Monthly Feedback Loop Model

This module implements a monthly time-series forecasting workflow with online learning and synthetic data generation. The functionality is now split into modular components across the project structure.

Architecture

The monthly feedback loop is organized into three main components:

1. **Ingestion Layer** (`ingestion/ingest_synthetic_production.py`)

Generates and ingests synthetic monthly production data.

- Fetches the latest 2 production records
- Generates synthetic data for the next month using trend analysis and seasonality factors
- Inserts the synthetic row into `gold.monthly_production`

2. **ETL/Modeling Layer** (`etl/gold_monthly_model.py`)

Trains forecasting and explanation models on historical data.

- Loads historical monthly production data
- Encodes months cyclically using sine/cosine features
- Trains `SGDRegressor` for online production forecasting
- Trains `SGDClassifier` for predicting explanation categories
- Generates predictions for the latest data point

3. **Agent/Orchestration Layer** (`agents/monthly_feedback_agent.py`)

Coordinates the entire feedback loop workflow.

- Ensures database tables exist (`gold.monthly_production`, `model_metrics`)
- Calls ingestion to generate synthetic data
- Calls model training and prediction
- Records model metrics and explanations to the database

PROF

What it does

The complete monthly feedback loop performs the following:

1. Connects to the Neon PostgreSQL database using repo utilities
2. Generates a synthetic production row for the next month based on trends and seasonality
3. Trains online forecasting models on historical data
4. Predicts production and generates a learned explanation reason
5. Stores metrics and predictions in `model_metrics` table

Key features

- **Modular design**: Each component can be run independently or as part of the workflow
- Synthetic data generation with trend analysis and Gaussian noise
- Incremental/online forecasting with `SGDRegressor`
- Learned explanation categories with `SGDClassifier`
- Cyclical month encoding for seasonality patterns
- Flexible database integration using existing `utils.db` helpers

Required environment

- Python with dependencies from the repo `requirements.txt`
- `NEON_DATABASE_URL` environment variable pointing to the Neon PostgreSQL database

How to run

From the repository root:

```
export NEON_DATABASE_URL="postgresql://<user>:<password>@<host>:
<port>/<database>"
```

Option 1: Run the complete feedback loop (Recommended)

```
python agents/monthly_feedback_agent.py
```

Option 2: Run individual components

Ingest synthetic data only:

```
python ingestion/ingest_synthetic_production.py
```

PROF

Train models only:

```
python etl/gold_monthly_model.py
```

Option 3: Run the original monolithic script

```
python energy_pipeline/models/monthly_feedback_loop.py
```

What is saved

- `gold.monthly_production`: New synthetic row for the next month

- `model_metrics`: Stores forecasted value, actual value, absolute error, and explanation reason

Database tables

The workflow requires/creates:

- `gold.monthly_production`: Historical and synthetic monthly production data
    - Columns: `year`, `month`, `month_name`, `total_production_gwh`, `avg_renewable_share_pct`, `growth_rate_pct`
- `model_metrics`: Model predictions and evaluation metrics
    - Columns: `year`, `month_name`, `predicted_gwh`, `actual_gwh`, `absolute_error`, `prediction_reason`, `created_at`

Notes

- The script assumes `gold.monthly_production` is already populated with historical rows
- If the database is empty, the script will raise an error
- Synthetic data generation uses seasonality factors and trend analysis for realistic values
- The explanation model identifies patterns like summer peaks, winter dips, growth trends, and renewable share impact
- Each component logs detailed information for monitoring and debugging